

Low Level C++ Theory & Applications

Amlal El Mahrouss (Meeting C++ 2026)

(Or how to write Modern C++ Software based on real examples)

I: **Warning**

- The slides are more of a supporting content for the talk, what I will do will be more of a orating lecture. This is so that this talk could share ideas between the speaker and its listeners.

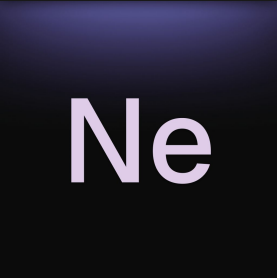
Email :
Web :

amlal@ne-app.eu
ne-app.eu

I: Observations

Amlal El Mahrouss

- When to use RAII?
- List of Several RAII Containers.
- Applications and Examples.
- Error Handling and Graceful Exits.

The logo for Ne, featuring the letters 'Ne' in a white, sans-serif font, centered within a dark blue square with a subtle gradient and a slight shadow effect.

Ne

II: When to use the RAII?

Amlal El Mahrouss

RAII is a crucial pattern introduced in Stepanov's STL papers, in this part of the lecture, we will study applications of the STL in areas that are non-trivial.

Where RAII shines is mostly where data really matters

- Distributed Systems
- Database Engines
- Operating Systems
- Firewall Software

As in its consistency and validation matters, and that we'd be able to fetch it correctly.

When using non-freestanding C++ RAII is mostly a ‘no-brainer’:

- `std::unique_ptr`.
- `std::shared_ptr`.

However, one may judge to implement its own container when deemed necessary, such as what the Open C++ Libraries ‘tracked_ptr’ does, or Boost’s custom expansions of shared pointers operations such as `delete_ptr`.

When to use RAII?

Real life examples:

Ant resource handling in HAL.dll and minoskrnl.exe:

- The architecture lets us use the RAII pattern to manage resources that are deemed to be eligible for such operations.
- Another example, file descriptors are when used by a routine are automatically freed after the routine finishes running.
- One more examples is that of DDK guards, a system that clears the locking guard before exiting the function.

Pool allocation in a PDF rendering software.

This is a non-trivial operation, where the complexity of the algorithm may be exponential here. Thus per RAI, we will need to perform caching and implement a pool allocation algorithm.

This helps for the following two concepts:

- Linearity of the search.
- Searching time per algorithm complexity.

III: List of Several RAII Containers

Amlal El Mahrouss

There is **OwnPtr**, A type introduced in **WebKit**, back when C++0x wasn't formalized yet.

Another example: Ne.app Ant and NeAnt have their own RAII primitives. Such as OwnPtr and Ref...

There also exists GCC extensions for that purpose as well:

Source: [Implementing RAII in pure C? - Stack Overflow](#)

IV: Applications and Examples

Amlal El Mahrouss

```
$ telnet telnet.example.com
```

Now... One shall handle, the TUI, resources allocation, logging, etc...

What happens if the resource is busy? What happens if the NIC is being plugged off? Or the router crashes?

That's why RAII containers such as `unique_lock` or `shared_lock` are crucial here, it helps handles those cases!

\$ telnet telnet.example.com (Made up server host)

One may wrap the resources in a ``Ref`` container and pointers into a ``RefPtr`` container. Thus giving us the ability to handle the telnet program resources more effectively.

```
$ telnet telnet.example.com (Made up server host)
```

One may wrap the resources into a ``Ref`` container and pointers into a ``RefPtr`` container. Thus giving us the ability to handle the telnet program resources more effectively.

However, abuse of RAII concepts and patterns, leads to issues with the program down the line. As it will become increasingly difficult to keep track of allocations and what's being freed and allocated.



```
VoidPtr x = MmAllocHeap(sz, flags, ...);  
if (x != nullptr) {  
    /// Turn 'x' in a pool or whatever...  
}
```

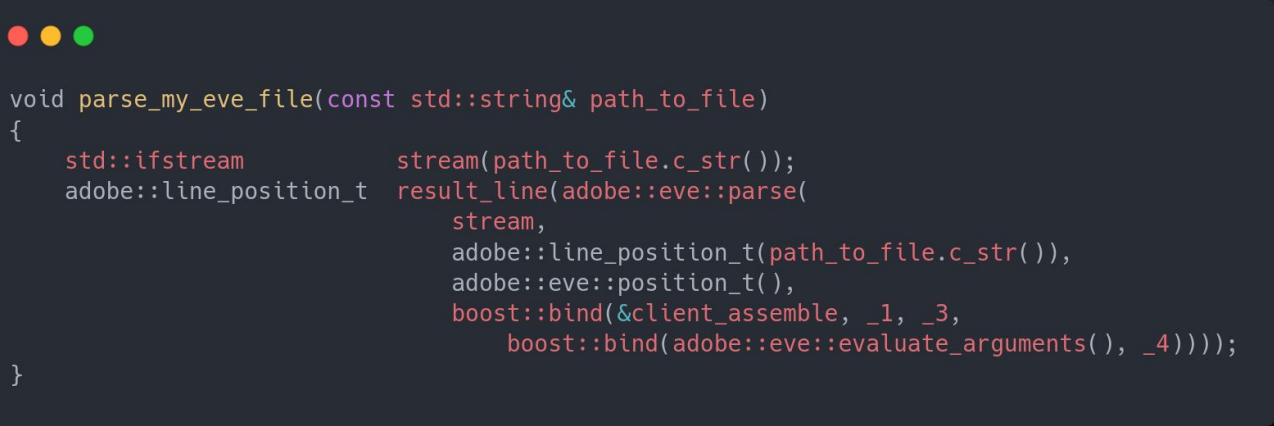


```
/// Now the x is freed when doing function unwinding on the stack.  
OwnPtr<VoidPtr> x = MmAllocHeap(sz, flags, ...);  
if (x) {  
    /// Turn 'x' in a pool or whatever...  
}
```

*This applies well for this example, under a specific caveat: **The resource scope and lifetime has to be determined and predictable.***

An example of RAII with Adobe Eve:

(https://stlab.adobe.com/group_asl_tutorials_eve.html)



```
void parse_my_eve_file(const std::string& path_to_file)
{
    std::ifstream      stream(path_to_file.c_str());
    adobe::line_position_t result_line(adobe::eve::parse(
        stream,
        adobe::line_position_t(path_to_file.c_str()),
        adobe::eve::position_t(),
        boost::bind(&client_assemble, _1, _3,
                    boost::bind(adobe::eve::evaluate_arguments(), _4))));
}
```

Note: One may provide a deleter predicate on the container initialization for allocators that have non-trivial behavior and are not compatible with the standard C++ allocator. E.g: COM, Qt...

[1]

<https://stackoverflow.com/questions/9308862/memory-management-in-com>

V: Error Handling and Graceful Exiting

Amlal El Mahrouss

Per Jason Turner's Talk on **Library Design...**

One shall think **about error handling in the infancy of the program's development...**

This should be present in the early days of the library.

[1]

https://www.youtube.com/watch?v=zL-vn_pGGgY

Thank you for listening!

There is slides available on my site:

amlal.nekernel.org (or the mirror pub.ne-app.eu)

Questions?

Amlal El Mahrouss